# Detecting Machine-Generated Text in Low-Resource South African Nguni Languages

Umar Parker
University of Cape Town
Cape Town, South Africa
PRKUMA001@myuct.ac.za

## Abstract

The rapid growth of large language models (LLMs) has resulted in machine-generated text (MGT) being harder to distinguish from human-written text. This has raised concerns about misinformation, academic integrity and content authenticity. While much research has been done on MGT detection, this is largely confined to high-resource languages such as English. There is relatively little research done on low-resource languages such as South African languages. This research investigates MGT detection for two South African languages: isiXhosa and isiZulu. We construct a filtered corpus from FineWeb-2 to create a language-adaptive pretraining (LAPT) [4] pool and a separate pool of reserved human-written text for fine-tuning. We generate machine-written text using Gemma 3 4B IT [12] and filter it for quality. We combine this machine-generated text with the reserved human-written pool to create a labeled classification dataset for fine-tuning a binary classification head for MGT detection. We fine-tune multilingual transformer models, mBERT [6] and XLM-R [5], using low-rank adaptation (LoRA) [14], and compare this against a logistic regression baseline. We found that LAPT benefits detector accuracy but depends on the model used as a backbone, LAPT method and the language scope. Both XLM-R and mBERT showed significant improvements over their no-LAPT baselines with the configurations for each model differing. We also showed that LoRA uses less peak GPU memory while training substantially fewer parameters than full fine-tuning but showed reduced robustness to a punctuation-based confound check.

## 1 Introduction

The rapid growth of large language models (LLMs) has made machine-generated text (MGT) increasingly difficult to detect. LLMs can generate fluent text in high-resource languages such as English making the differences between MGT and human-written text a lot more difficult to detect compared to earlier generations of language models. This is a concern as MGT can be used to spread misinformation, undermine academic integrity and lead to a lack of trust in online resources across the internet. This makes reliably detecting machine-generated text an important part of safeguarding content authenticity.

This has caused MGT detection to become a highly active field of research. This has resulted in the development of benchmarks, such as RAID [8]. RAID is a challenging benchmark that has shown that even state-of-the-art detectors struggle to generalise across unknown domains and generators. However, most of the research on detecting MGT, including RAID, relates to English. Multilingual benchmarks such as M4 [29] and MULTITuDE [21] extend detection to other languages, but only cover a small number of high-resourced languages. These benchmarks also lack coverage for African languages and more specifically South African languages. This pattern reflects the broader imbalance in NLP research where the majority of languages do not receive attention in state-of-the-art NLP research [15, 23]. For isiXhosa and isiZulu specifically, this problem is compounded by the morphological structures of these languages. Both languages are agglutinative languages that are written conjunctively with rich noun-classes [28]. This means that words are formed by stringing together morphemes. Since the subword tokenisers used by multilingual models are trained mostly on English data, they may fragment these morphologically dense words poorly, which may reduce the quality of machine-generated text and impair detectors' abilities to identify it.

This leads to the question we investigate in this work: How well does a language-adaptively pretrained and LoRA fine-tuned XLM-RoBERTa (XLM-R) and mBERT models compare to a basic logistic regression model when detecting machine-generated text across isiXhosa and isiZulu, measured by macro-F1 alongside accuracy, precision, recall, and AUC-ROC.

To investigate this, we construct a filtered corpus for isiXhosa and isiZulu derived from FineWeb-2 [25]. We split the corpus into a language-adaptive pretraining pool and a reserved pool of human-written text. We then create a MGT detection dataset by generating text for each reserved human-written text using Gemma 3 4B IT [12], filtering each for quality before creating human-machine text pairs. We then fine-tune mBERT [6] and XLM-R [5] transformer models. We test multiple configurations both including and excluding LAPT [4] and LoRA [14] and compare the results to a logistic regression baseline following related work on adapting multilingual models to African languages [2]. We evaluate these models both in-domain and per-language for isiXhosa and isiZulu. In summary (1) we construct a filtered, language-adaptive pretraining corpus and reserved human-written pool for isiXhosa and isiZulu derived from FineWeb-2 (2) a labeled MGT detection dataset by generating machine-written counterparts using Gemma 3 4B IT and a filtered reserved pool (3) LAPT and LoRA fine-tuned mBERT and XLM-R classifiers compared to a logistic regression baseline (4) an empirical comparison across multiple configurations, evaluated in-domain and per-language.

Section 2 covers the related works. Section 3 covers the data pipeline used filter and create the MGT dataset, Section 4 covers how the models were pretrained and fine-tuned, Section 5 covers the evaluation metrics used and the post-hoc confound checks done, Section 6 presents the results and discussion with Section 7 concluding the paper.

## 2 Related Works

This section outlines previous work conducted on topics important for our research. First we discuss current approaches to MGT detection and benchmarks that are used to evaluate them. We then cover the multilingual models we use in this work and their pretraining objective. Next we look at domain- and language-adaptive pretraining and parameter-efficient fine-tuning, namely LoRA, as methods for adapting multilingual models for new languages and domains. Lastly we discuss multilingual and cross-domain generalisation in MGT detection. Each of these areas are actively researched and we intend on jointly applying them to isiXhosa and isiZulu.

### 2.1 Machine-Generated Text Detection

There are three main methods of MGT detection. These are zero-shot statistical detection, watermarking and supervised classification.

*zero-shot statistical detection:* Zero-shot methods use the log-probabilities of a model or similar model suspected of generating a text to check whether that text is machine generated. These methods do not require gathering a dataset of real vs machine text, specifically training a classifier or using a watermark. DetectGPT [22] is an example of this type of method that analyses the model's log-probability function. Text generated by LLMs tend to lie in the areas of negative curvature of the model's log-probability function. This is due to how models select tokens. Models select tokens based on their own learned probability distribution. This causes models to generate text that lies near or on local maxima of the model's log-probability function. Perturbations in the text represent a shift from the local maximum point. This causes the perturbed versions to have a lower log-probability than the original generated text. Since human-written text is more randomly distributed across the model's probability landscape, perturbations of human-written text do not show a consistent drop in log-probability. DetectGPT generates perturbations of the text by using a generic mask-filling model that has not been fine-tuned on the target domain. Multiple rewrites of the text are generated. The log-probability is then calculated for the original text and all the perturbed text. The difference between the log-probability of the original text and the average log-probabilities of the perturbed text is computed. If the discrepancy is higher than a set decision threshold, then the text is classified as machine-generated text. The advantages of this method are that it generalises better to new domains where supervised detectors fail since it uses the generative model itself to score the text. It also remains accurate even if the text is changed by a human writer. The downsides of this method are that it requires the token-level log-probabilities to be known which is not the case for many closed-source models and it is a computationally heavy method that requires more time for detection than other methods.

*Watermarking:* Watermarking methods aim to embed a statistical signal into a text at generation that is not identifiable by a human but is detectable by an algorithm. Kirchenbauer et al. [17] propose a framework that aims to do this by splitting a model's vocabulary into a list of "green" tokens and "red" tokens using a pseudo-random hash of the tokens used as a seed of a random number generator. The "green" tokens are tokens that the model is allowed to use and the "red" tokens are restricted. The model is biased to choose "green" tokens over "red". Kirchenbauer et al. implement two versions of this, one where the "red" tokens are banned and one where a bias parameter is added to the logits of the "green" tokens before converting them to probabilities. Using the first method results in great degradation of the output text while the second method has minimal impact on the text assuming that the scenario is high-entropy. The biggest downside of watermarking methods in general is that it has to be done at generation time and cannot be added afterwards to a text from an unknown source.

*supervised classification:* Supervised classification relies on training a model using labeled text to be able to classify whether it is human or machine text. Zellers et al. [31] show that a discriminative classifier is able to accurately detect fake news when it is built on the same architecture as the generator. Grover uses top-p sampling when generating text. This means that token choices are restricted to the smallest set of tokens whose cumulative probability exceeds a threshold p. This means that low probability word choices that occur in human-written text are avoided by the model. This distinguishes the human-written text from the machine-generated text. When a classification head is added on top of the final hidden state, it is able to accurately determine if text is generated by Grover since it shares the same architecture and vocabulary distribution which allows it to detect if a text stays in the top-p subset. This statistical signature of machine-generation is more difficult to detect if an unrelated architecture is used. However, this same advantage limits its use in real-world applications since multiple generators exist and can be used to generate fake news which makes classifiers tuned to one generator a poor choice due to classifiers generalising poorly to text from another generator.

Most MGT detection research is focused on English and applying detection to multilingual environments is discussed in Section 2.5.

### 2.2 Multilingual Transformer Models

Both models used in this work are pretrained using Masked Language Modelling (MLM), where random tokens in an input sequence are replaced with a mask token. The model is then trained to predict the original token based on the surrounding context. Devlin et al.[6] introduce this objective as part of BERT, which extends to mBERT, a model pretrained jointly on Wikipedia text across 104 languages using a single shared vocabulary and MLM objective without any cross-lingual supervision. Conneau et al. [5] extend this approach to XLM-RoBERTa (XLM-R), pretraining the model on a much larger filtered CommonCrawl corpus spanning 100 languages using MLM alone and shows that cross-lingual pretraining at sufficient scale allows a single shared model to perform competitively with monolingual models. This is also true for low-resource languages that do not have enough resources to create a dedicated model. Since both XLM-R and mBERT use the same MLM pretraining objective, it can be continued on new unlabelled text after pretraining.

### 2.3 Domain- and Language-Adaptive Pretraining

Domain-adaptive pretraining (DAPT) continues unsupervised pretraining on unlabeled text from a target domain before task-specific

fine-tuning. Gururangan et al. [13] show that DAPT improves downstream task performance across a range of domains, even with a small amount of in-domain data.

Language-adaptive pretraining (LAPT) [4] is a process that adapts the principles of DAPT to multilingual settings. The difference between DAPT and LAPT is that LAPT treats the variety of languages as the different domains. LAPT is relevant to low-resource languages as it allows pretraining on unlabeled in-language data which is easier to find than labelled corpora. Alabi et al. [2] use LAPT to do continued pre-training on African languages, adapting multilingual models such as XLM-R on monolingual African language text. Alabi et al. report substantial downstream gains even with these languages being underrepresented in the models' original pretraining data. However, their evaluation focuses on tasks such as named entity recognition and topic classification and does not consider MGT detection, making the gains transfer to this task unknown.

## 2.4 Parameter-Efficient Fine-Tuning

Fully fine-tuning large pre-trained models is computationally expensive. Low-rank adaptation (LoRA) [14] addresses this by freezing the pretrained model weights and injecting small trainable low-rank matrices into each transformer layer. This reduces the trainable parameters while performing comparably when compared to full fine-tuning on many tasks. This makes LoRA well suited to resource constrained environments such as running multiple LAPT and fine-tuning configurations which was done in this work.

## 2.5 Multilingual and Cross-Domain Generalisation

Benchmarks such as RAID [8] have been developed to evaluate detectors under realistic conditions. RAID consists of over 6 million generations that span 11 models, 8 domains, 11 adversarial attacks and 4 decoding strategies. To generate the texts, a chat and non-chat template were created. The chat template is for models fine-tuned on dialogue and the non-chat template is used for base completion models. The templates contain a dynamic title placeholder that is substituted with the title of human-written document at generation. The 11 adversarial attacks are applied to simulate an adversary trying to bypass detection. RAID's experiments show that state-of-the-art detectors struggle to generalise to unseen domains and generators. In this work we use RAID's continuation construction strategy when generating the human/machine text pairs which we will describe further in Section 3.

Apart from RAID and other English focused benchmarks, multiple efforts have been made to extend MGT detection to multilingual settings. Wang et al. [29] introduce M4 , a benchmark that spans multiple languages, generators and domains. Wang et al. show that detectors trained on only one domain or generator often fail to generalise to unseen ones. Macko et al. [21] similarly introduce MULTITuDE, a benchmark covering eleven languages generated by eight multilingual LLMs, motivated by the observation that most existing detection benchmarks are purely English only and cover a limited set of older models. Both benchmarks expand language coverage but do not include African or specifically South African languages. Both focus on relatively well-resourced languages. This leaves a gap in understanding in how MGT detectors perform for morphologically rich, low-resource languages such as isiXhosa and isiZulu. This is the gap this work aims to address.

# 3 Data Pipeline

In this section we aim to discuss how we constructed our dataset. This includes how our initial corpus was filtered, how our LAPT and reserved pool were constructed and how we generated our text for our dataset.

## 3.1 Corpus filtering pipeline

The dataset that we used for this project was created using FineWeb-2 [25]. FineWeb-2 is a multilingual web-text corpus that is an extension of the original FineWeb [24]. FineWeb-2 extended the FineWeb dataset to the multilingual setting. The core motivation of FineWeb-2 was to create a filtering process that automatically adapts its processing, language identification thresholds and quality filtering heuristics based on the statistical properties of the languages as apposed to the previous method of using one uniform set of rules for all languages. FineWeb-2 covers 1,868 language-script pairs. The corpus is split into a "train" and "test" set and uses the language configurations of "xho_Latn" for isiXhosa and "zul_Latn" for isiZulu. Our main justification of using FineWeb-2 was its relatively large amount of data combined with the ability to filter by scraped date and language quality.

*3.1.1 Score and year filtering:* The first step of creating our corpus was to concatenate the two sets into one larger set. This was due to the fact that we needed as much data as possible and that we would do our own filtering. The second step was to filter the combined corpus by the date it was crawled and the language score of the crawled text. We chose a cut-off score of 0.95 (only accepted text with a language_score >= 0.95) to make sure that the texts used were the correct language. This was important since some texts with low confidence were included and in some cases these were entirely in English. RefinedWeb [26] included this language score filtering before moving onto more expensive filtering processes. Our year cut-off was to ensure that we used texts that were highly unlikely to be generated from an LLM to prevent model collapse [27]. Our choice of using 2021 as a cut-off was a methodologically conservative choice due to generators still struggling to generate text in isiXhosa and isiZulu.

While this filtering removed a large amount of English text from the dataset, It was not a perfect solution since one domain namely "staffbull.msk.ru" was able to bypass the language score check since some of the texts were scored over 0.95 but were all in English. Another issue was that the filtering of FineWeb-2 scored many documents that were made up of ID-style codes, such as license plate numbers, highly. We manually removed "staffbull.msk.ru" from the dataset and ensured that the ID-style codes were filtered out.

*3.1.2 Domain extraction and domain-frequency filtering:* To get the domain of documents, we used the URL of the document. This was used throughout the filtering pipeline for two purposes: a minimum-frequency filter and a maximum-concentration cap.

The minimum-frequency filter was used to remove domains that appeared less than the threshold amount. We reasoned that domains that appeared less than the threshold were difficult to judge as reliable sources and could be low-quality scrapes that were able to pass the filtering process of FineWeb-2. Initially this was set at 10 but ultimately we used 5 as the cut-off number. This almost doubled the representation of unique domains and increased the corpus value by 3.7%. This added long-tail diversity without reducing quality.

We implemented a maximum-concentration cap due to the fact that unfiltered web-scraped corpora suffers from domain concentration which can negatively affect downstream tasks. Wettig et al. [30] shows that the Zipfian (long-tail) shape of the domain-frequency distributions has a few domains that contribute the majority of texts while the majority of domains only provide a few documents to the overall pool of texts. The paper also argues that domain-level curation is a legitimate method to increase dataset quality alongside quality filtering. This phenomena was found in our data after filtering by language score and year. We found that "xh.airbnb.com" made about 61% of the documents in the corpus after filtering the language score and year. We report the domain-concentration in the Appendix following the practise of analysis used in RefinedWeb [26].

We also filtered out adult-content. This was done since we noticed that domains referring to adult content showed up in our document pools. We used a word-boundary regex to flag domains suspected of adult-content. We then used an LLM to translate and check if the domains contained adult content and manually verified if the domain needed to be blacklisted. The LLM was used since we are not speakers of the languages. In the future it would be more reliable to have a speaker review the content and refine the regex to make sure filtering is accurate. We confirmed and blocked 16 adult-content domains and applied this filtering to both pools.

Lastly we removed a translation-memory lookup database from the dataset. We found this when we were checking for English content mislabelled as isiXhosa or isiZulu and found that a domain containing text in multiple languages (English, isiXhosa and Chinese) was present. This domain was "mymemory.translated.net" which is a public translation-memory lookup database. This meant that it evaded our filter, as it contained genuine isiXhosa and isiZulu words, just not connected prose. We found that only 8 documents were excluded when filtering out this domain. We discuss the pools created in 3.2.

*3.1.3 Near-duplicate and exact-duplicate filtering.* We applied two deduplication methods to the dataset. We applied minHash near-duplicate clustering and exact deduplication. Duplicate documents make models perform worse [18]. In the case of a classifier, it can result in inflated scores due to document leakage where some training data is duplicated in the test and validation set. We filtered documents using FineWeb-2's precomputed minhash_cluster_size field, excluding documents that belonged to a near-duplicate cluster larger than our threshold of 50.

We applied exact deduplication afterwards to ensure that there were no exact copies in the dataset to prevent the documents from leaking to the test set and inflating the test accuracy. There were duplicates due to CommonCrawl crawling the same URL over time, content being published across multiple different domains and multiple URLs pointing to the same content.

*3.1.4 Length and prose-content filtering.* Our last two filters applied to the dataset was a length filter and a prose-content filter. For the length filter, we discarded documents that contain less than 200 characters of text since this would not be enough content for continued pretraining or for the continued generation. The prose-content filter aimed to filter out content that consists of transaction code strings (UA1KA UA1KB UA1KC...) which was found during manual review. This content passes the filtering process of FineWeb-2 as it is long and has a high language score. The filter measured how many of the "words" follow this code style pattern compared to real words and rejected documents that pass a threshold of 30%.

## 3.2 LAPT and reserved pool construction

Once the corpus was filtered, it was then split into two disjoint pools: a LAPT pool of unlabelled text and a reserved pool which was used for text generation. The reserved corpus was created by taking 10% of the available documents from the previously filtered dataset. The main corpus underwent another filtering process to determine the eligibility of documents to be selected for the reserved pool.

We excluded certain domain patterns that are known machine-translation. These domains tend to follow a pattern where a 2-3 letter language code is followed by a period with the parent domain afterwards. We created a regex to filter out these domains (2-3 letter language code.root domain) and manually removed a few that showed up when reviewing the domains. We also removed religious content domains due to them most likely being translated text rather than originally created. We tried to ensure that the texts used were most likely real human-written text due to the fact that the machine translated text could be distinguishable from the human-written text allowing a classifier to learn translated vs originally-composed instead of human vs machine text.

We also applied a domain concentration cap of max 15% independently to both the LAPT and reserved pool. This was to prevent one domain from dominating the data while still benefiting from a percentage of the data. This was critical to diversify the pools as one example of a dominating domain was "isolezwelesixhosa.co.za" reaching 48% of the reserved pool before the domain cap was introduced. This directly reflects the concern documented by RefinedWeb's [26] domain-analysis practice and the domain re-balancing done in "Organize the Web" [30]. It also follows FineWeb-2's finding that rebalanced datasets improve performance. It should be noted, however, that FineWeb-2's dataset re-balancing focused on duplication count instead of domain frequency. When applying the max domain cap, we made sure to iterate multiple types over the dataset to make sure that the percentage of a domain was always close to 15% even if the dataset shrinks. This was especially true when constructing the reserved pool.

The reserved pool documents were all given a "seed_id" based on the id field of FineWeb-2. This was to check that each document was unique using an automated uniqueness check.

## 3.3 Machine-text generation

The strategy we adapted to generate the text is the one used in RAID [8]. Instead of using a dynamic title, We used some text

from the chosen document as context which we kept dynamic as "{context}" in the prompt template. The generator we used was Gemma 3 4B IT [12] due to the fact that we were able to freely access it with a huggingFace account. One drawback of using an open-source model like Gemma was that it struggled to generate natural continuations with the prompt instructions we gave it. We will discuss the outputs of the generator more in Section 6.

The process we followed to generate all the MGT was to take each document from the reserved pool and split it into a shared context with two continuations, one human and one machine generated. The context length was from 40-100 words with a human continuation having a length of 70-150 words. The generated continuation targeted a length equal to that of the human continuation. We targeted equal lengths to prevent it from becoming a signal for the classifier to detect. We used multiple domains to diversify the amount of training data to help a trained classifier generalise better following rationale from both M4 [29] and MULTITuDE [21] which both found that detector generalisation depends on the training and evaluation data spanning multiple domains and, in MULTITuDE's case, multiple languages evaluated separately. We used this as justification to keep the evaluation of isiXhosa and isiZulu separately.

The prompt consisted of multiple sentences that describe what the model should do. We initially told the model that it was a native speaker of either isiXhosa or isiZulu to persuade the model to generate text as fluently as possible without language switching. We then provided the context of the document to the model specifying that it continued in the style used by the document. This was to make sure the styles of the continuations were the same. This was important since an early iteration of the prompt without this restriction allowed classifiers to learn the mismatch of styles as an indicator of MGT. We then listed requirements that had to be in the generated text, such as text length. One requirement worth noting is the command to not restate or rephrase information already given and to introduce new details to advance the topic. This was included to prevent the generator from paraphrasing the text but introduced and unintentional behaviour that caused the model to introduce new qualitative detail but avoid adding concrete numeric specifics. We retained this behaviour due to not knowing if this vagueness is caused by the property of the LLM used.

We used Google Colab to generate the human/machine pairs. We ran it on the A100 GPU available.

## 3.4 Generation Quality Control

We filtered the generated text before we accepted them. The first thing we filtered for was prompt-artifacts. We would reject all generations that included any instruction texts, commentary and formatting placeholders. Secondly, we filtered for degenerate repetition. We used a periodic-repeat algorithm to make sure that a phrase of any length would not repeat 3 times or more times consecutively. This was found to be flawed since afterwards it was found that phrases would still be repeated but spaced out in the text which would not be detected by the algorithm. Thirdly, we checked that the generator was not copying context directly by using a longest-common-word-run check and rejecting generations that failed the check. Lastly we used GlotLID [16], a language identification model designed for broad coverage of low-resource languages, to check that the generated text is in the correct language.

## 3.5 Data Splitting

Before we did LAPT pretraining and then fine-tuning the classifiers, we did one last filtering of the LAPT pool and created test, validation and test sets for fine-tuning. The last filtering step we did for LAPT was to add all the documents from the reserved pool that was not able to be used for generation due to previous filtering. We did this since it still passed LAPT filtering just not filtering for the reserved pool. Once added back to the LAPT pool, we checked that there were no overlaps between the pools and there were no new duplicates in the LAPT pool. We used this as our final LAPT pool which was then chunked for pretraining. The total amount of documents used for LAPT were 80,446 with 42,588 documents from isiXhosa and 37,858 from isiZulu.

For the train, validation and test sets, we split it with 80% for training and 10% for validation and testing. This resulted in the train set containing 9,374 pairs with isiXhosa having 4,307 of these pairs and isiZulu having 5,067 pairs, the validation set having 538 isiXhosa pairs and 633 isiZulu pairs totalling 1,171 pairs and the test set containing 539 isiXhosa pairs and 634 isiZulu pairs totalling 1,173 pairs. In total, there were 11,718 pairs. When constructing the splits, we grouped the documents by seed id to ensure that human and machine-generated pairs always stayed in the same split. We also preserved the proportions of isiXhosa and isiZulu across the splits using MULTITuDE's [21] precedent for multilingual MGT benchmark construction.

The token counts are given in Table 3 in the Appendix.

# 4 Modeling

In this section, we discuss our LAPT pipeline design, the TF-IDF baselines and the classifier fine-tuning design.

## 4.1 LAPT Pipeline Design

*4.1.1 Data preparation.* Both models used, mBERT and XLM-RoBERTa (XLM-R), have a hard architectural limit on how many tokens they can process in a single forward pass. This is a value fixed at 512 tokens for both. This made using the LAPT pool as is a bad idea since many web docs contained more than 512 tokens. To allow the model to take in all the tokens available, we needed to split the text into chunks which could be fed to the models without any text being truncated.

mBERT and XLM-R have different vocabularies that are incompatible. mBERT uses Wordpiece tokeniser and XLM-R uses SentencePiece tokeniser. This means that we have to do separate chunking for mBERT and XLM-R. When the data was chunked, it was packed into 512-token blocks using the FULL-SENTENCES approach [20]. A separator token was inserted at the document boundary. When we chunked the data, we grouped them by language to ensure that isiXhosa and isiZulu text did not get mixed mid-sequence.

*4.1.2 Pretraining.* For pretraining, we had two training modes. One mode was full-parameter LAPT and the other was LoRA LAPT using a rank of 256. We also pretrained on one language instead of both to evaluate the cross-lingual performance of the models. The

configurations we had were: full-parameter pretraining, LoRA pretraining and full-parameter pretraining on isiXhosa only or isiZulu only. We used these configurations for both mBERT and XLM-R. We trained 8 models following these configurations. We used fixed hyperparameters using Gururangan et al.'s [13] values rather than searching for the best parameter and used 3 epochs mainly due to a tight compute budget. During the runs, we logged the time it took to train the model, the peak GPU memory, the throughput and the total trainable parameters. A pre-LAPT perplexity score was calculated once per run before training was done to have a baseline to compare to after LAPT was completed.

We trained the mBERT models locally using an RX 7700XT and the ROCm software stack on Ubuntu 26.04. We ran the XLM-R models on Google Colab using the A100 GPU. XLM-R's larger vocabulary made it much slower to run locally and use more VRAM making it more sensible to run on Google Colab.

## 4.2 TF-IDF baselines

We created two variants of the baseline, where we use word-level TF-IDF and char-level TF-IDF. We added char-level TF-IDF due to the agglutinative nature of the languages. Word-level n-grams do not share the sub-word structure of words that share the same root. This is a limitation of the languages where a root can take many forms by prefixing.

We tuned the models' hyperparameters via randomised search as opposed to grid search [3] over 20 iterations. This was cheaper to run due to the size of the hyperparameter space while not exhaustively searching over the space. The hyperparameter search was validated on PredefinedSplit not k-fold cross-validation to match the transformer baselines. The best configuration was refit on the training data.

## 4.3 Classifier Fine-Tuning Design

We tested 10 configurations of fine-tuning for both mBERT and XLM-R. This consisted of a full-fine-tuning run for each pretrained model and a LoRA fine-tuning run for each model. We then also tested full-fine-tuning and LoRA fine-tuning for both models without using LAPT (2 runs with no LAPT, 2 Runs with full-pretraining LAPT, 2 runs with LoRA pretraining, 2 runs with full isiXhosa LAPT, 2 runs with full isiZulu LAPT to make the ten runs).

To get the hyperparameters, we used BOHB-style search [10] using Optuna's TPESampler [1]. This strategy uses Bayesian optimisation. Bayesian optimisation builds a probabilistic model on the best regions of hyperparameter spaces tend to perform well and use this model to infer the next configuration to use rather than sampling blindly. With this, hyperbandPruner [19] monitors the validation accuracy of a trial and prunes trials that are underperforming before they complete all their epochs. When a LoRA trial was run, rank was also searched rather than fixed.

Each model ran 8 trials before completing. All configurations were run on an AMD RX 7700XT locally using the ROCm software stack.

# 5 Evaluation

In this section we discuss the evaluation metrics used and the post-hoc confound checks we conducted.

## 5.1 Evaluation Metrics

Each classifier run reports the accuracy, precision, recall, F1 score, confusion matrix and RAID calibrated results. This was done for the TF-IDF baseline and both mBERT and XLM-R classifiers. We also did pairwise McNemar's tests [7] at both the default and RAID-calibrated thresholds, Bonferroni-corrected [9] across the comparisons. McNemar's tests compare two classifiers evaluated on the same test set looking at examples where they disagree. The test checks whether a classifier wins a disagreement substantially more than the 50/50 split that would be expected of two equally good classifiers. The p-value produced by this test is the probability of seeing a disagreement split as extreme as the one observed, assuming that both classifiers are equally good. A small p-value indicates that an extreme split is unlikely to occur by chance alone. This provides evidence that there is a difference between classifiers. Since we conducted roughly 100 comparisons, we applied Bonferroni correction. This divides the significance thresholds by the number of tests performed to reduce the risks of false positives appearing due to the number of tests done. This produced a corrected threshold of $\alpha = 0.0005$. Many non-significant comparisons in our results had very few discordant pairs due to the near-ceiling accuracy observed and should be treated as inconclusive due to limited statistical power rather than a confirmation of equivalence.

## 5.2 Post-hoc validation checks

We ran four checks after the completed results. These were a length-confound check, a punctuation and formatting robustness check, a repetition confound check and a linguistic feature confound check.

*5.2.1 Length-confound check:* This was done to ensure that the classifiers trained were not relying on the length of the MGT to determine if it was machine-generated text. It was found that the sentence accumulation would stop once a minimum word count was hit due to a bug in the generation script not caught while generating the text. This made the length distribution skew between human and machine text. To test this, we used a classifier trained to only look at word-count and found that it could still get 65.71% accuracy. To reduce this, we constructed a distribution-matched subset binned by the word count and subsampled per bin to make the length distributions as close to equal as possible. We found that using the same word count classifier, it only got an accuracy of 51.98%. We then tested the fine-tuned models on this length-matched subset and found that the accuracy barely moved indicating that the classifiers were not relying on the length-confound to achieve their accuracy.

*5.2.2 Punctuation and formatting robustness check.* A Punctuation and formatting robustness check was run after the length-confound check. This was run due to generated text seeming to struggle with punctuation especially for non-prose text. We found that eleven punctuation count features (newlines, commas, colons, semicolons, asterisks, hashes, quotes, leading list-marker flags) produced an accuracy of 76.85% on the raw text set. This is larger than the length confound. It was found that the human text contained more newlines and semicolons than the machine text, while the machine text had more asterisks, commas and numbered-list starts. The newline and list-markers could potentially be due to the instruction for the generator to continue in the style of the context given.

We applied the control in two stages. The first stage only stripped structural artifacts such as newlines, leading list markers, asterisks, hashes, while leaving the core grammatical punctuation untouched. This was kept as they could represent a stylistic difference as opposed to just being a generation artifact. This stage only reduced the diagnostic's accuracy to 73.10% showing that this accuracy was not mainly due to generation artifacts. The second stage also removed semicolons, colons and comas. This reduced the diagnostic accuracy to 50.09% showing that these punctuation were the cause of the high diagnostic accuracy.

We evaluated the classifiers and TF-IDF baselines on these new subsets of data, stage 1 filtering and stage 2 filtering. We did not retune the models on this data. This approach follows RAID [8] where they perturbed test-time inputs against a fixed detector. It was found that this only dropped the accuracy by a few percentage indicating that a different learned signal is responsible for the accuracy of the classifiers. It was also found that punctuation made no difference to the accuracy of the TF-IDF baseline results.

*5.2.3 Repetition confound check.* This was done to determine if there were many repeated phrases that could easily be detected by the classifier. During generation, we used a periodic-repeat algorithm to reject text that had repeating phrases. This only caught phrases that were repeated three or more times consecutively. After we did the classifier fine-tuning tests, we created a new repetition detector that looked for 5-word n-grams that were repeated three or more times in the text. This was more thorough as it looked for phrases throughout the text instead of only consecutive phrases. We only scanned the machine-generated text and removed them, along with their human text pairing, from the test set. We then tested all the classifiers on this new test set. We found that out of all the test documents, only 6.05% were flagged out of 1 173 documents available. In total, 71 documents were flagged with 38 coming from isiXhosa (7.05% of 539) and 33 coming from isiZulu (5.21% of 634). The rest of the results are shown in Section 6.6.

*5.2.4 Linguistic feature confound check.* This check was done after it was found that our baseline TF-IDF achieved a near-ceiling accuracy with specific words being indicators of machine text. Our aim was to see how much these linguistic features influenced detectability. To do this, we constructed a lexicon based on the POS corpus created by Gaustad and Puttkammer [11]. This corpus contains tokens in isiXhosa and isiZulu with a label that indicates its POS. We focused on tokens that had the "CONJ" tag since the types of conjunctions used were suspected to be a strong indicator of machine text. We then counted the lemma frequency only keeping ones that appeared three or more times throughout the corpus.

We created a logistic regression diagnostic classifier that computed three features. These features were machine-leaning connectives, other connectives and infinitive-verbs. We split connectives into two features since we found that three words, ukuba, ukuthi and ngoba, were strongly machine-leaning according to our baseline TF-IDF coefficients while the rest were human-leaning. For the infinitive-verb feature, we took tokens that start with the "uku" prefix rather than a word list since this captures infinitive forms of tokens in the corpus rather than only the ones shown in the TF-IDF top-feature output. We excluded previously confirmed connectives from this count to avoid double-counting and feature contamination. We calculated each feature as a rate per 100 words.

We created a distribution-matched subset by using the diagnostic classifier's own fitted coefficients, which were found by training the classifier on the original train split. We combined these coefficients into one score and used it to split the documents into ten bins, with equal-width score ranges, based on the scores. We then subsampled the documents to the smaller count of the documents to equalise the class counts. Unlike the previous checks, this matched subset does not keep the human and machine pairs since matching per-document scores required treating the two classes as independent pools. The test set contained 520 documents that had equal amounts of human-written text (260 for each) and machine-generated text. We then evaluated the trained classifiers on this matched subset without recalibration.

**Table 1: TF-IDF and no LAPT baselines with the highest full LAPT and LoRA config for mBERT and XLM-R using the RAID accuracy.**

| Model | Configuration | Default Acc. | RAID Acc. | Achieved FPR |
|---|---|---|---|---|
| TF-IDF (word) | – | 0.9974 | 0.5000* | 0.0000 |
| TF-IDF (char) | – | 0.9974 | 0.5000* | 0.0000 |
| mBERT | No LAPT, full FT | 0.9974 | 0.9757 | 0.0486 |
| mBERT | Full LAPT (xho-only), full FT | 0.9983 | 0.9680 | 0.0639 |
| mBERT | LoRA LAPT (combined), full FT | 0.9991 | 0.9970 | 0.0051 |
| XLM-R | No LAPT, full FT | 0.9987 | 0.9783 | 0.0435 |
| XLM-R | Full LAPT (xho-only), full FT | 0.9996 | 0.9940 | 0.0119 |
| XLM-R | LoRA LAPT (combined), full FT | 0.9996 | 0.9804 | 0.0392 |

*Threshold unreachable, see Section 6.1 for explanation.

## 6 Results and Discussion

This section covers the results of the classification performance, the effect of LAPT on the accuracy, the effect of LoRA on fine-tuning and the cross-lingual results. We found that the default threshold accuracy was uninformative as all the accuracy had numbers of over 99% accuracy leading us to look at RAID-calibrated accuracy that shows the difference in configurations. These are the sections we cover: Overall results of the models and TF-IDF baselines, the LAPT effect for both full-pretraining and LoRA pretraining, LoRA vs full fine-tuning, per-languages and cross-lingual performance, significance testing, robustness checks and the final discussion.

### 6.1 Overall Classification Performance

Our first finding was that across all 22 models tested, we had accuracy figures of 0.9970-0.9996. All the models' accuracy is saturated near-ceiling across all models. This likely indicates that the generation of text introduces a lot of artifacts that are easily detectable by the models. Our prompt (Section 3.3) forced the model to continue text without paraphrasing or copying text, leading it to generate text that did not contain specific details about the context. Gemma would not include numeric specifics if needed and it would struggle with natural continuation sometimes repeating phrases in the text multiple times. Because of this, we look at RAID-calibrated accuracy with a target of 5% false positive rate (FPR). We found that the spread of the RAID-calibrated accuracy was between 0.9680 and 0.9970 for the transformer models with the TF-IDF models both

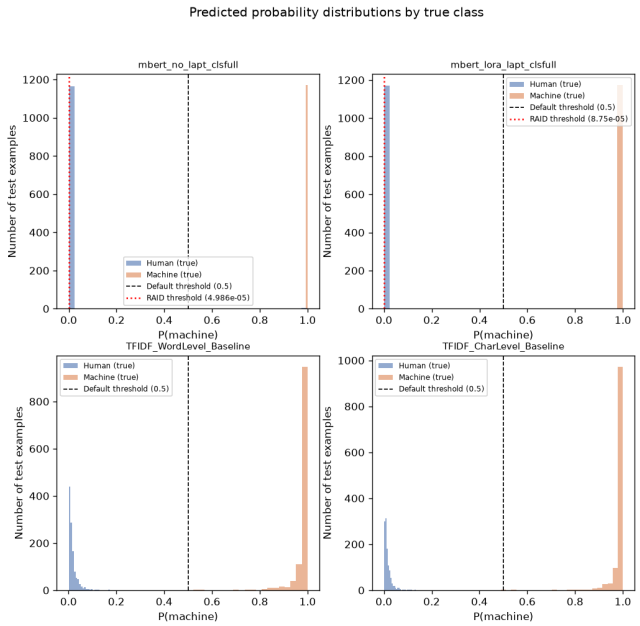



**Figure 1: The predicted probability distribution for two mBERT configurations (no-LAPT, LoRA LAPT) and the TF-IDF baselines (word-level, char-level) with default and RAID-calibrated thresholds marked.**

having an accuracy of 0.5 since it does not have fine-grained threshold tuning. TF-IDF predicted probabilities are heavily concentrated around 0 and 1 without enough examples in between probability values (Figure 1) for the threshold search to identify a cut-off that achieves 5% FPR. This causes the threshold search to fail and the model is reported at chance-level accuracy. We found that the best RAID-calibrated performer was XLM-R using full LAPT on isiXhosa only with full fine-tuning with a score of 0.9940 and an FPR of 0.0119. The worst scoring model was mBERT with full LAPT on isiXhosa only with full fine-tuning having a score of 0.9680 and a FPR of 0.0639. Notably, both the best and worst model use the same configuration, full LAPT on isiXhosa only with full fine-tuning, suggesting mBERT and XLM-R respond very differently to this LAPT condition.

## 6.2 Effect of Language-Adaptive Pretraining

We found that LAPT's effect on the overall accuracy of a model is highly dependent on the configuration and backbone model used. Our strongest case of this was XLM-R with full LAPT on isiXhosa only outperforms XLM-R's no-LAPT baseline with the RAID-calibrated accuracy of 0.9940 vs 0.9783 ($p \approx 1.9 \times 10^{-8}$). The same was found to be true for mBERT. mBERT with LoRA LAPT outperforms mBERT's no-LAPT 0.9970 vs 0.9757 ($p \approx 5.8 \times 10^{-11}$). In this case, the LoRA LAPT also beats the full LAPT ($p \approx 1.19 \times 10^{-9}$). When comparing XLM-R to mBERT using the same configuration (full LAPT on isiXhosa only), we found that XLM-R benefitted more than mBERT ($p \approx 2.2 \times 10^{-12}$). We also found that mBERT with LoRA LAPT benefitted more from this than XLM-R ($p \approx 6.8 \times 10^{-9}$).

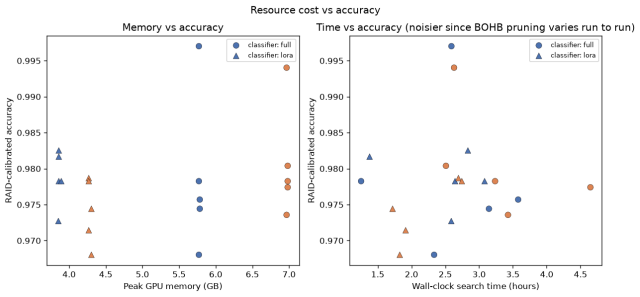



**Figure 2: Peak GPU memory and wall-clock search time vs. RAID-calibrated accuracy, comparing full fine-tuning and LoRA classifier configurations across both backbones.**

Most of the comparisons did not provide significant results. This was mainly shown when comparing a full LAPT isiZulu only model. However, many of these comparisons had few discordant pairs, indicating limited statistical power rather than equivalence between conditions. Overall we found that LAPT sometimes improves MGT detection for isiXhosa and isiZulu but is dependant on the backbone model, language-scope and LAPT-method combination and requires more research before a conclusive statement can be made. The 12 significant comparisons are in Table 8.

## 6.3 Effect of LoRA vs. Full Fine-Tuning

It was found that only three out of 10 pairs showed a statistically significant difference. These three pairs were mBERT LoRA LAPT, mBERT full LAPT with isiXhosa only and XLM-R LAPT on isiXhosa only. mBERT LoRA LAPT displayed a full fine-tuning RAID accuracy of 0.9970 vs LoRA fine-tuning's 0.9783 with a p-value of $p \approx 5.4 \times 10^{-10}$ in favour of full fine-tuning. mBERT with full LAPT on isiXhosa only showed a full fine-tuning RAID accuracy of 0.9680 vs 0.9817 for LoRA fine-tuning with a p-value of 0.0003 favouring LoRA. XLM-R full LAPT with isiXhosa only had a full fine-tuning accuracy of 0.9940 vs LoRA fine-tuning's 0.9744 with a p-value of $p \approx 1.8 \times 10^{-9}$ favouring full fine-tuning. The other 7 pairs do not show any significant difference, however, accuracy is not the only metric measured.

We measured the resource cost of full fine-tuning vs LoRA fine-tuning. LoRA fine-tunes fewer parameters than full fine-tuning, approximately only 0.17-1.3% of the full parameter count. LoRA fine-tuning also uses less VRAM with full fine-tuning using 5.77-6.98 GB of VRAM compared to LoRA fine-tuning's 3.85-4.30 GB. The reduction is roughly 33-39% depending on the backbone model.

While LoRA trains less parameters and uses less VRAM, it is less robust than full fine-tuning. We found that when we stripped the punctuation, LoRA was affected more greatly than full fine-tuning in 9 out of 10 pairs with the only exception being mBERT with no-LAPT. Notably, the mBERT full LAPT with isiXhosa configuration, where LoRA classifier fine-tuning significantly outperformed full fine-tuning on RAID-calibrated accuracy, also shows the largest robustness gap of all ten pairs, with a 4.82 percentage-point accuracy drop for LoRA vs 0.30 for full fine-tuning under punctuation stripping. This suggests that accuracy alone would favour LoRA here, while a deeper robustness check reveals a hidden cost.

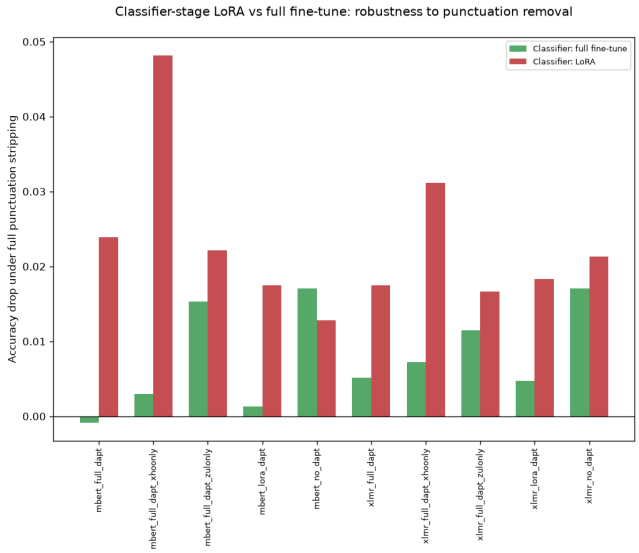



**Figure 3: Accuracy drop under full punctuation stripping for matched full fine-tune vs. LoRA classifier pairs, across all 10 configurations.**

## 6.4 Per-Language and Cross-Lingual Results

While Section 6.2 examined how LAPT language scope affects overall model performance, this section examines whether models detect isiXhosa and isiZulu text with equal accuracy. Unlike sections 6.2 and 6.3, these per-language differences were not formally tested for statistical significance and are descriptively reported. We found that 18 of the 20 transformer models favour isiXhosa with only 2 favouring isiZulu. This is especially true for XLM-R with all 10 XLM-R configurations favouring isiXhosa with gaps ranging from 0.59-2.64 percentage points. The two exceptions to this were mBERT full LAPT with isiXhosa only and full classifier fine-tuning and mBERT full LAPT with isiZulu only and full classifier fine-tuning. mBERT full LAPT with isiXhosa and full classifier fine-tuning achieved a RAID accuracy of 0.9601 for isiXhosa vs isiZulu's 0.9748 even though it was language-adapted to isiXhosa which is quite counter-intuitive. mBERT full LAPT with isiZulu only and full classifier fine-tuning had an isiXhosa RAID accuracy of 0.9879 vs isiZulu's 0.9629 which is the opposite of what is expected. These results suggest that LAPT applied to a specific language does not necessarily improve the detection accuracy of the same language.

## 6.5 Statistical Significance

This section looks at the entire body of significance testing with the 12 significant results shown in Table 8 in the Appendix. We found that there were 0 out of 100 significant comparisons at the default threshold. When using the RAID-calibrated threshold, however, we found 12 significant comparisons out of 100 total comparisons. In these 12 comparisons, significant results show up in every category tested (LAPT vs no-LAPT, full vs LoRA LAPT, cross-lingual comparisons, backbone model comparisons and classifier method comparisons). This suggests that RAID-calibrated metrics are consistently sensitive across different types of comparisons. Many non-significant comparisons involve few discordant pairs, given the near-ceiling accuracy observed throughout, and should be interpreted with the same statistical power caveat in Section 5.1.

## 6.6 Robustness and Confound Checks

In this section we look at the punctuation, length robustness, repetition and linguistic feature confound checks. Starting with the punctuation checks, we found that using a diagnostic classifier on the raw test set got an accuracy of 76.85%. When we did stage 1 filtering mentioned in Section 5.2.2, the accuracy fell slightly to 73.10%. When we applied stage 2 filtering, the accuracy dropped to 50.09%. Stage 1 filtering showed that structural formatting alone only accounted for a small portion of the detected signal, while the majority of the diagnostic's performance came specifically from the comma, colon and semicolon frequency. We found that the TF-IDF models were structurally immune to these filtering. For the word-level TF-IDF we got a -0.004pp change and for the char-level TF-IDF, we got a +0.08pp drop. The changes for transformer models ranged from -0.09pp-4.82pp across all 20 configurations. This is much less of an impact than the diagnostic's collapse showing that the classifiers are not relying on punctuation for detection. It is worth noting that these drops are not uniformly distributed across the models. These variations were covered in detail in Section 6.3.

With the length-confound check, we found that a word-count-only diagnostic classifier obtained an accuracy of 65.71% before we accounted for the possible length differences in the human text and MGT. This showed a real length confound existed in the data. Once we matched the length of the texts, the accuracy of the classifier dropped to 51.98%, confirming that the matched subset is a valid control. When we re-evaluated the 20 transformer configurations on the matched subset with default threshold accuracy, we found that the biggest difference was -0.148pp for XLM-R full-LAPT with isiXhosa only and the LoRA classifier fine-tuning. The RAID-calibrated accuracy only changed by at most 0.50pp across the entire matrix, showing that there was no meaningful degradation due to the length mismatches.

For the repetition confound check, we found that only 71 out of 1,173 documents (6.05%) contained multiple repeated phrases. We tested all 22 models on this new test set. We found that the default-threshold range was only between -0.02pp and +0.03pp for the transformer models compared to the original accuracy. This indicates that removing these repetitive documents did not make any meaningful difference. For our RAID-calibrated accuracy, we had changes in the range of -0.04pp to +0.32pp across all the transformers with the largest change for XLM-R with full LAPT on isiZulu only and LoRA fine-tuning. For our TF-IDF baselines, we found no changes in the performance of the classifiers. This was due to the fact that the threshold is unreachable which we discussed in Section 6.1. These results showed that while repetition is a detectable artifact, the generator did not produce a lot of it and that it was not the main driver of the classifiers' accuracy.

In our language features confound check, we found that a logistic regression diagnostic classifier, using the three features mentioned in Section 5.2.4, was able to achieve an accuracy of 88.92%. This was much higher than the previous checks where the word-count diagnostic classifier achieved an accuracy of 65.71% and the

punctuation-count diagnostic achieved 76.85%. This showed that the usage of connectives and infinitive verbs are a good signal for machine text. When we reevaluated the 20 transformer models on the distribution-matched dataset, we found that, for 19 of the 20 models, there was a mean percentage point drop of around -1.4pp. The largest drop was -2.76pp, with only one model showing an increase. Unlike the previous confound checks, there was some indication that the transformer classifiers used these as a detection signal.

### 6.7 Lexical Indicator Analysis

In this section we focus on the type of words and characters that were used as indicators for human and machine text. The linear structure of TF-IDF allowed us to get an exact coefficient for each word and n-gram which we used to create a diagnostic classifier and a matched subset mentioned in Section 5.2.4. We did not do this for mBERT and XLM-R since they do not have fixed per-word coefficients. We found that conjunctions used in the text were a good signal for detecting human or machine text. We found that some conjunctions were human-leaning such as xa (when), uma (if), nangona (although) and ukuze (in order to). The three highest machine-leaning conjunctions that we found were ukuba (if/to be), ukuthi (that) and ngoba (because). Other machine indicators were infinitive verb forms and generic vocabulary such as abantu (people) and esihle (beautiful). Many top machine features shared the uku-infinitive prefix. This pattern was repeated by the char-level baseline, where the top machine indicators were pieces of the same infinitive forms (ukuba, ukub, kuba, hamba). This showed that human text contained more complex sentence structure and vocabulary compared to the simple and generic text generated by the machine. We also found that human text contained some code-switching with English words, such as "the", appearing as a human indicator whereas the generator was restricted to only using words from isiXhosa or isiZulu. One thing to note is that the generic nature of the machine text could be due to the generator avoiding specific values when asked to generate new text instead of paraphrasing. Whether these lexical patterns are learned by the transformer models cannot be tested by the coefficient extraction since they do not have fixed per-word weights but the diagnostic classifier and matched-subset reevaluation in Section 6.6 suggest they share at least a part of this signal.

### 6.8 Discussion

In answer to this paper's research question, LAPT can sometimes significantly improve MGT detection for isiXhosa and isiZulu, but the effect is dependent on the backbone model, language-scope and LAPT-method rather than a general rule. TF-IDF models were able to achieve a raw accuracy of 99.74%. This suggests the task may not require deep linguistic understanding. The detection may be catching shallow artifacts of Gemma 3 4B IT's generation rather than testing genuinely sophisticated content-based distinction. This does not mean that detection for isiXhosa and isiZulu is easy but rather that the generator, which is small compared to many frontier models, was not able to produce text without clear detectable signals. Specifically, Section 6.7 found that generated text contained infinitive-heavy and generic vocabulary compared to the complex sentences included in human text while also leaving out code-switching which was present in the human text. This pattern was validated by a diagnostic classifier which reached 88.92% accuracy with only linguistic features and was shown to extend to the mBERT and XLM-R models. We also found that XLM-R and mBERT respond differently to LAPT configurations. This could be due to the pre-training corpus composition or due to the models using different tokenizers. The difference in the vocabulary sizes could also impact the results of LAPT. We also found that on raw accuracy, LoRA sometimes matches full fine-tuning or beats it rarely (Section 6.3). This however is not true for robustness as LoRA performed worse than full fine-tuning when punctuation was stripped. We also found that isiXhosa outperformed isiZulu except for two cases mentioned in Section 6.4 that were counter-intuitive.

The limitations faced in this work were that we only used one model for generation. Using more than one model would allow the detectors to be more diverse and would allow us to compare the outputs of generations from different models. We could then compare the generation quality of the models. We also did not test multiple different prompting strategies and prompts, with our prompt causing the model to generate text that contained generic vocabulary and lacked sentence complexity. We also excluded cross-domain evaluation making the detector less useful in real scenarios. We also only used isiXhosa and isiZulu not the broader Nguni languages or other South African language families.

Extensions of this project would be to test with multiple generators including larger models. We would extend the classifiers to Sesotho, Setswana or other Nguni languages, we would also compute cross-domain evaluation and revisit the prompt used for generation. We could also look into the different responses of the backbone models to LAPT.

## 7 Conclusions

This work investigated how language-adaptive pretraining and using LoRA for fine-tuning could compare to a logistic regression model when detecting machine-generated text for isiXhosa and isiZulu. We found that LAPT can significantly improve MGT detection for isiXhosa and isiZulu but the effect depends on the model used (mBERT, XLM-R), LAPT method and language scope. We also found that using LoRA for fine-tuning reduced the amount of parameters that were trained and the VRAM usage, but full fine-tuning showed a small robustness advantage. We also found that specific linguistic features were genuine indicators of machine-generated text. Our contributions made were a filtered FineWeb-2 derived corpus, a labeled MGT dataset that consisted of human and machine pairs, the LAPT/LoRA evaluation pipeline and the confound-robustness checks. This work is limited by its reliance on a single generator and two languages, discussed further in Section 6.8. Future work would be to extend this work to additional Nguni languages, generators and evaluating the detectors in cross-domain settings (Section 6.8).

## References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-Generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. Association for Computing Machinery, 2623–2631. doi:10.1145/3292500.3330701

[2] Jesujoba O. Alabi, David Ifeoluwa Adelani, Marius Mosbach, and Dietrich Klakow. 2022. Adapting Pre-trained Language Models to African Languages via Multilingual Adaptive Fine-Tuning. In *Proceedings of the 29th International Conference on Computational Linguistics*. International Committee on Computational Linguistics, Gyeongju, Republic of Korea, 4336–4349. https://aclanthology.org/2022.coling-1.382/

[3] James Bergstra and Yoshua Bengio. 2012. Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research* 13, 10 (2012), 281–305. https://jmlr.org/papers/v13/bergstra12a.html

[4] Ethan C. Chau, Lucy H. Lin, and Noah A. Smith. 2020. Parsing with Multilingual BERT, a Small Corpus, and a Small Treebank. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, Online, 1324–1334. doi:10.18653/v1/2020.findings-emnlp.118

[5] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised Cross-lingual Representation Learning at Scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 8440–8451. doi:10.18653/v1/2020.acl-main.747

[6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. https://arxiv.org/abs/1810.04805

[7] Thomas G. Dietterich. 1998. Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms. *Neural Computation* 10, 7 (1998), 1895–1923. doi:10.1162/089976698300017197

[8] Liam Dugan, Alyssa Hwang, Filip Trhlík, Andrew Zhu, Josh Magnus Ludan, Hainiu Xu, Daphne Ippolito, and Chris Callison-Burch. 2024. RAID: A Shared Benchmark for Robust Evaluation of Machine-Generated Text Detectors. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 12463–12492. https://aclanthology.org/2024.acl-long.674/

[9] Olive Jean Dunn. 1961. Multiple Comparisons Among Means. *J. Amer. Statist. Assoc.* 56, 293 (1961), 52–64. doi:10.1080/01621459.1961.10482090

[10] Stefan Falkner, Aaron Klein, and Frank Hutter. 2018. BOHB: Robust and Efficient Hyperparameter Optimization at Scale. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*. PMLR, 1437–1446. https://proceedings.mlr.press/v80/falkner18a.html

[11] Tanja Gaustad and Martin J. Puttkammer. 2022. Linguistically Annotated Dataset for Four Official South African Languages With a Conjunctive Orthography: IsiNdebele, IsiXhosa, IsiZulu, and Siswati. *Data in Brief* 41 (2022), 107994. doi:10.1016/j.dib.2022.107994

[12] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. 2025. Gemma 3 Technical Report. *arXiv preprint arXiv:2503.19786* (2025). https://arxiv.org/abs/2503.19786

[13] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. 2020. Don't Stop Pretraining: Adapt Language Models to Domains and Tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 8342–8360. doi:10.18653/v1/2020.acl-main.740

[14] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685* (2021). https://arxiv.org/abs/2106.09685

[15] Pratik Joshi, Sebastin Santy, Amar Budhiraja, Kalika Bali, and Monojit Choudhury. 2020. The State and Fate of Linguistic Diversity and Inclusion in the NLP World. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 6282–6293. doi:10.18653/v1/2020.acl-main.560

[16] Amir Hossein Kargaran, Ayyoob Imani, François Yvon, and Hinrich Schütze. 2023. GlotLID: Language Identification for Low-Resource Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, Singapore, 6155–6218. https://aclanthology.org/2023.findings-emnlp.410/

[17] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A Watermark for Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 17061–17084. https://proceedings.mlr.press/v202/kirchenbauer23a.html

[18] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. 2022. Deduplicating Training Data Makes Language Models Better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Dublin, Ireland, 8424–8445. doi:10.18653/v1/2022.acl-long.577

[19] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. 2018. Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization. *Journal of Machine Learning Research* 18, 185 (2018), 1–52. https://jmlr.org/papers/v18/16-558.html

[20] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019). https://arxiv.org/abs/1907.11692

[21] Dominik Macko, Robert Moro, Adaku Uchendu, Jason Samuel Lucas, Michiharu Yamashita, Matúš Pikuliak, Ivan Srba, Thai Le, Dongwon Lee, Jakub Simko, and Maria Bielikova. 2023. MULTITuDE: Large-Scale Multilingual Machine-Generated Text Detection Benchmark. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 9960–9987. doi:10.18653/v1/2023.emnlp-main.616

[22] Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D. Manning, and Chelsea Finn. 2023. DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 24950–24962. https://proceedings.mlr.press/v202/mitchell23a.html

[23] Wilhelmina Nekoto, Vukosi Marivate, Tshinondiwa Matsila, Timi Fasubaa, Taiwo Fagbohungbe, Solomon Oluwole Akinola, Shamsuddeen Hassan Muhammad, Salomon Kabongo Kabenamualu, Salomey Osei, et al. 2020. Participatory Research for Low-Resourced Machine Translation: A Case Study in African Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, Online, 2144–2160. doi:10.18653/v1/2020.findings-emnlp.195

[24] Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. 2024. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In *Advances in Neural Information Processing Systems*, Vol. 37. 30811–30849.

[25] Guilherme Penedo, Hynek Kydlíček, Vinko Sabolčec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargaran, Colin Raffel, Martin Jaggi, Leandro Von Werra, and Thomas Wolf. 2025. FineWeb2: One Pipeline to Scale Them All – Adapting Pre-Training Data Processing to Every Language. *arXiv preprint arXiv:2506.20920* (2025). https://arxiv.org/abs/2506.20920 Preprint, under review.

[26] Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Hamza Alobeidli, Alessandro Cappelli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. 2023. The RefinedWeb Dataset for Falcon LLM: Outperforming Curated Corpora with Web Data Only. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023) Datasets and Benchmarks Track*. https://openreview.net/forum?id=kM5eGcdCzq

[27] Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. 2024. AI Models Collapse When Trained on Recursively Generated Data. *Nature* 631 (2024), 755–759. doi:10.1038/s41586-024-07566-y

[28] Elsabé Taljard and Sonja E. Bosch. 2006. A Comparison of Approaches to Word Class Tagging: Disjunctively vs. Conjunctively Written Bantu Languages. *Nordic Journal of African Studies* 15, 4 (2006), 428–442.

[29] Yuxia Wang, Jonibek Mansurov, Petar Ivanov, Jinyan Su, Artem Shelmanov, Akim Tsvigun, Chenxi Whitehouse, Osama Mohammed Afzal, Tarek Mahmoud, Toru Sasaki, Thomas Arnold, Alham Fikri Aji, Nizar Habash, Iryna Gurevych, and Preslav Nakov. 2024. M4: Multi-generator, Multi-domain, and Multi-lingual Black-Box Machine-Generated Text Detection. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, St. Julian's, Malta, 1369–1407. https://aclanthology.org/2024.eacl-long.83/

[30] Alexander Wettig, Kyle Lo, Sewon Min, Hannaneh Hajishirzi, Danqi Chen, and Luca Soldaini. 2025. Organize the Web: Constructing Domains Enhances Pre-Training Data Curation. In *Proceedings of the 42nd International Conference on Machine Learning*. https://arxiv.org/abs/2502.10341

[31] Rowan Zellers, Ari Holtzman, Hannah Rashkin, Yonatan Bisk, Ali Farhadi, Franziska Roesner, and Yejin Choi. 2019. Defending Against Neural Fake News. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc., 9054–9065. https://papers.nips.cc/paper/9106-defending-against-neural-fake-news


## 8 AI Usage Statement

The AI tool used was Claude. It was used to plan out the Sections and headings. It was also used for feedback on drafts specifically looking at grammar, argument validity and content accuracy with explanations to help search up relevant information. It was also used to format tables and graphs in overleaf. We used the feedback to edit the text and fix any inaccuracies present. We also used it for translating isiXhosa and isiZulu text. The AI tool was also used

to format results in an easy to understand way. The writing in the paper was our own work with AI influencing revisions but made after reviewing it.

## 9 Supplementary Information

**Listing 1: Generation prompt template for isiXhosa. The isiZulu template is structurally identical, with the target language name substituted throughout (*isiXhosa → isiZulu*).**

```
You are a native isiXhosa speaker.

Below is the beginning of a document.

<context>
{context}
</context>

Continue this document naturally,
in the same style and format as the context.

Requirements:

- Write ONLY in isiXhosa.
- Do NOT copy any sentences from the context.
- Do NOT simply restate or
  rephrase information already given in the
  context - introduce new, specific details
  that advance the topic,
  narrative, or argument.
- Do NOT translate.
- Do NOT explain what you are doing.
- Do NOT include the word "Translation".
- If the context reads like a list, article,
  or promotional page, continue in
  that same format - headings,
  lists, or other formatting are fine if they
  fit naturally as a continuation.
- Produce natural, fluent isiXhosa.
- Length: approximately 70-150 words.
```

### 9.1 Examples of Code-Switching in Corpus Data

Here are some examples of isiXhosa and isiZulu text containing some English words and phrases.

(1) **isiXhosa** (`seed_id: xho_d0ad9b9a`):
…Ngenxa yokuba izixeko ezifana yaseVenice, eRoma, kunye Paris izigidi abakhenkethi ukuya eYurophu minyaka yonke. Siye wadala uluhlu engambi… Newsletter. **Ukufuna iihotele and more** … Posts zakutsha…

(2) **isiZulu** (`seed_id: zul_d8c29abb`):
Isihloko eyisihluthulelo Canavero lalingu inqubo uthemba ukwenza **in the next 24** izinyanga, alibiza ngokuthi ikhanda Anatomosis Venture, noma “Heaven”…

(3) **isiZulu** (`seed_id: zul_4eb62c7b`):
…Esikhathini salesi sihloko, Igor Gamanenko, **the** i-Semalt Umphathi Wempumelelo Yekhasimende, utshela indlela yokuvikela iwebhusayithi yakho ekuhlaselweni kwe-cyber…

(4) **isiXhosa** (`seed_id: xho_9b1c406a`):
…Ukwenza oku zonke bit eziphathekayo-bobabini ngaphezu koko liye lahlala efanayo iyintoni itshintshile-makhe siqwalasele **Michael Restivo and jaon van de Rijt** ngayo (2012)…

(5) **isiZulu** (`seed_id: zul_825b63e9`):
…I RIAA uyawuthanda koMthetho Digital Millenium Copyright (DMCA) uhlelo, kodwa ingabe bakuthola kuyakhungathekisa. Where iseluleko Google kusho into eyodwa, **the** RIAA wathatha line eziphikisayo…

**Table 2: Top 10 domains by document count in the final filtered corpus (LAPT pool and reserved pool combined, $n = 92{,}164$), after all filtering stages including the 15% domain-concentration cap.**

| Rank | Domain | Documents | % of total |
|---|---|---|---|
| 1 | xh.airbnb.com | 6,204 | 6.73% |
| 2 | isolezwelesixhosa.co.za | 6,131 | 6.65% |
| 3 | zu.unansea.com | 5,324 | 5.78% |
| 4 | jw.org | 3,241 | 3.52% |
| 5 | isolezwe.co.za | 2,464 | 2.67% |
| 6 | xh.eferrit.com | 2,057 | 2.23% |
| 7 | news24.com | 2,048 | 2.22% |
| 8 | xh.martech.zone | 1,932 | 2.10% |
| 9 | zu.martech.zone | 1,700 | 1.84% |
| 10 | isizulu.news24.com | 1,585 | 1.72% |

Together, the top 10 domains account for 35.47% of the combined corpus. No domain exceeds the 15% domain-concentration cap described in Section 3.2.

**Table 3: Document and token counts for the LAPT pool and the train, validation and test splits. These counts are per-row (one row per human or machine documents e.g. the train split's 9,374 human/machine pairs correspond to 18,748 rows). WordPiece and SentencePiece token counts are computed using the mBERT and XLM-R tokenisers respectively, without special tokens.**

| Split | isiXhosa | isiZulu | Total docs | WordPiece tok. | SentencePiece tok. |
|---|---|---|---|---|---|
| LAPT pool | 42,588 | 37,858 | 80,446 | 118,649,898 | 99,265,682 |
| Train | 8,614 | 10,134 | 18,748 | 4,874,364 | 3,960,448 |
| Validation | 1,076 | 1,266 | 2,342 | 608,689 | 494,855 |
| Test | 1,078 | 1,268 | 2,346 | 611,226 | 496,321 |
| Total | 53,356 | 50,526 | 103,882 | 124,744,177 | 104,217,306 |

**Table 4: Full results across all 22 evaluated configurations.**

| Backbone | LAPT status | Language scope | Classifier | Accuracy | Precision | Recall | F1 | AUC-ROC | RAID Acc. |
|---|---|---|---|---|---|---|---|---|---|
| mBERT | Full LAPT | Combined | Full | 0.9983 | 0.9974 | 0.9991 | 0.9983 | 0.99999 | 0.9783 |
| mBERT | Full LAPT | Combined | LoRA | 0.9970 | 0.9949 | 0.9991 | 0.9970 | 0.99998 | 0.9825 |
| mBERT | Full LAPT | isiXhosa only | Full | 0.9983 | 0.9983 | 0.9983 | 0.9983 | 0.99999 | 0.9680 |
| mBERT | Full LAPT | isiXhosa only | LoRA | 0.9983 | 0.9983 | 0.9983 | 0.9983 | 0.99994 | 0.9817 |
| mBERT | Full LAPT | isiZulu only | Full | 0.9974 | 0.9966 | 0.9983 | 0.9974 | 0.99998 | 0.9744 |
| mBERT | Full LAPT | isiZulu only | LoRA | 0.9970 | 0.9941 | 1.0000 | 0.9970 | 0.99998 | 0.9783 |
| mBERT | LoRA LAPT | Combined | Full | 0.9991 | 0.9991 | 0.9991 | 0.9991 | 0.99957 | 0.9970 |
| mBERT | LoRA LAPT | Combined | LoRA | 0.9979 | 0.9966 | 0.9991 | 0.9979 | 0.99996 | 0.9783 |
| mBERT | No LAPT | n/a | Full | 0.9974 | 0.9949 | 1.0000 | 0.9974 | 0.99997 | 0.9757 |
| mBERT | No LAPT | n/a | LoRA | 0.9974 | 0.9958 | 0.9991 | 0.9974 | 0.99993 | 0.9727 |
| XLM-R | Full LAPT | Combined | Full | 0.9996 | 0.9991 | 1.0000 | 0.9996 | 0.99999 | 0.9774 |
| XLM-R | Full LAPT | Combined | LoRA | 0.9979 | 0.9974 | 0.9983 | 0.9979 | 0.99997 | 0.9680 |
| XLM-R | Full LAPT | isiXhosa only | Full | 0.9996 | 0.9991 | 1.0000 | 0.9996 | 0.99999 | 0.9940 |
| XLM-R | Full LAPT | isiXhosa only | LoRA | 0.9970 | 0.9949 | 0.9991 | 0.9970 | 0.99953 | 0.9744 |
| XLM-R | Full LAPT | isiZulu only | Full | 0.9987 | 0.9983 | 0.9991 | 0.9987 | 0.99999 | 0.9736 |
| XLM-R | Full LAPT | isiZulu only | LoRA | 0.9983 | 0.9974 | 0.9991 | 0.9983 | 0.99998 | 0.9714 |
| XLM-R | LoRA LAPT | Combined | Full | 0.9996 | 0.9991 | 1.0000 | 0.9996 | 0.99999 | 0.9804 |
| XLM-R | LoRA LAPT | Combined | LoRA | 0.9979 | 0.9966 | 0.9991 | 0.9979 | 0.99996 | 0.9787 |
| XLM-R | No LAPT | n/a | Full | 0.9987 | 0.9974 | 1.0000 | 0.9987 | 0.99998 | 0.9783 |
| XLM-R | No LAPT | n/a | LoRA | 0.9974 | 0.9974 | 0.9974 | 0.9974 | 0.99991 | 0.9783 |
| TF-IDF (char) | – | – | – | 0.9974 | – | – | 0.9974 | 0.99997 | 0.5000* |
| TF-IDF (word) | – | – | – | 0.9974 | – | – | 0.9974 | 0.9999 | 0.5000* |

*Threshold unreachable at 5% target FPR. see Section 6.1.

**Table 5: Full test set vs. length-matched-subset accuracy for all 20 transformer classifier configurations, at both the default (0.5) and RAID-calibrated thresholds. Differences are reported in percentage points (pp).**

| Configuration | Orig. Acc. | Matched Acc. | Diff (pp) | Orig. RAID Acc. | Matched RAID Acc. | RAID Diff (pp) |
|---|---|---|---|---|---|---|
| mBERT, full LAPT (combined), full FT | 0.9983 | 0.9981 | −0.02 | 0.9783 | 0.9770 | −0.12 |
| mBERT, full LAPT (combined), LoRA FT | 0.9970 | 0.9968 | −0.02 | 0.9825 | 0.9841 | +0.15 |
| mBERT, full LAPT (xho-only), full FT | 0.9983 | 0.9981 | −0.02 | 0.9680 | 0.9719 | +0.39 |
| mBERT, full LAPT (xho-only), LoRA FT | 0.9983 | 0.9981 | −0.02 | 0.9817 | 0.9821 | +0.05 |
| mBERT, full LAPT (zul-only), full FT | 0.9974 | 0.9968 | −0.06 | 0.9744 | 0.9739 | −0.06 |
| mBERT, full LAPT (zul-only), LoRA FT | 0.9970 | 0.9968 | −0.02 | 0.9783 | 0.9796 | +0.13 |
| mBERT, LoRA LAPT (combined), full FT | 0.9991 | 0.9987 | −0.04 | 0.9970 | 0.9968 | −0.02 |
| mBERT, LoRA LAPT (combined), LoRA FT | 0.9979 | 0.9981 | +0.02 | 0.9783 | 0.9802 | +0.20 |
| mBERT, no LAPT, full FT | 0.9974 | 0.9974 | +0.00 | 0.9757 | 0.9770 | +0.13 |
| mBERT, no LAPT, LoRA FT | 0.9974 | 0.9968 | −0.06 | 0.9727 | 0.9777 | +0.50 |
| XLM-R, full LAPT (combined), full FT | 0.9996 | 0.9994 | −0.02 | 0.9774 | 0.9751 | −0.23 |
| XLM-R, full LAPT (combined), LoRA FT | 0.9979 | 0.9974 | −0.04 | 0.9680 | 0.9643 | −0.37 |
| XLM-R, full LAPT (xho-only), full FT | 0.9996 | 0.9994 | −0.02 | 0.9940 | 0.9930 | −0.10 |
| XLM-R, full LAPT (xho-only), LoRA FT | 0.9970 | 0.9955 | **−0.15** | 0.9744 | 0.9707 | **−0.38** |
| XLM-R, full LAPT (zul-only), full FT | 0.9987 | 0.9987 | +0.00 | 0.9736 | 0.9719 | −0.16 |
| XLM-R, full LAPT (zul-only), LoRA FT | 0.9983 | 0.9974 | −0.08 | 0.9714 | 0.9751 | +0.37 |
| XLM-R, LoRA LAPT (combined), full FT | 0.9996 | 0.9994 | −0.02 | 0.9804 | 0.9770 | −0.34 |
| XLM-R, LoRA LAPT (combined), LoRA FT | 0.9979 | 0.9968 | −0.11 | 0.9787 | 0.9764 | −0.23 |
| XLM-R, no LAPT, full FT | 0.9987 | 0.9987 | +0.00 | 0.9783 | 0.9783 | +0.01 |
| XLM-R, no LAPT, LoRA FT | 0.9974 | 0.9968 | −0.06 | 0.9783 | 0.9764 | −0.19 |

All 20 configurations evaluated on the same length-matched subset ($n$ = 1,568), using each model's original decision threshold without recalibration. The largest change in each column is bolded.

**Table 6: Full test set vs. repetition-matched-subset accuracy for all 22 evaluated configurations, at both the default (0.5) and RAID-calibrated thresholds. Differences are reported in percentage points (pp).**

| Configuration | Orig. Acc. | Matched Acc. | Diff (pp) | Orig. RAID Acc. | Matched RAID Acc. | RAID Diff (pp) |
|---|---|---|---|---|---|---|
| mBERT, full LAPT (combined), full FT | 0.9983 | 0.9986 | +0.03 | 0.9783 | 0.9791 | +0.09 |
| mBERT, full LAPT (combined), LoRA FT | 0.9970 | 0.9968 | −0.02 | 0.9825 | 0.9832 | +0.07 |
| mBERT, full LAPT (xho-only), full FT | 0.9983 | 0.9982 | −0.01 | 0.9680 | 0.9691 | +0.11 |
| mBERT, full LAPT (xho-only), LoRA FT | 0.9983 | 0.9982 | −0.01 | 0.9817 | 0.9823 | +0.06 |
| mBERT, full LAPT (zul-only), full FT | 0.9974 | 0.9977 | +0.03 | 0.9744 | 0.9746 | +0.02 |
| mBERT, full LAPT (zul-only), LoRA FT | 0.9970 | 0.9973 | +0.03 | 0.9783 | 0.9791 | +0.09 |
| mBERT, LoRA LAPT (combined), full FT | 0.9991 | 0.9991 | −0.01 | 0.9970 | 0.9986 | +0.16 |
| mBERT, LoRA LAPT (combined), LoRA FT | 0.9979 | 0.9982 | +0.03 | 0.9783 | 0.9787 | +0.04 |
| mBERT, no LAPT, full FT | 0.9974 | 0.9977 | +0.03 | 0.9757 | 0.9764 | +0.07 |
| mBERT, no LAPT, LoRA FT | 0.9974 | 0.9977 | +0.03 | 0.9727 | 0.9732 | +0.05 |
| XLM-R, full LAPT (combined), full FT | 0.9996 | 0.9995 | −0.00 | 0.9774 | 0.9773 | −0.01 |
| XLM-R, full LAPT (combined), LoRA FT | 0.9979 | 0.9982 | +0.03 | 0.9680 | 0.9701 | +0.20 |
| XLM-R, full LAPT (xho-only), full FT | 0.9996 | 0.9995 | −0.00 | 0.9940 | 0.9936 | **−0.04** |
| XLM-R, full LAPT (xho-only), LoRA FT | 0.9970 | 0.9968 | −0.02 | 0.9744 | 0.9741 | −0.03 |
| XLM-R, full LAPT (zul-only), full FT | 0.9987 | 0.9986 | −0.01 | 0.9736 | 0.9750 | +0.15 |
| XLM-R, full LAPT (zul-only), LoRA FT | 0.9983 | 0.9982 | −0.01 | 0.9714 | 0.9746 | **+0.32** |
| XLM-R, LoRA LAPT (combined), full FT | 0.9996 | 0.9995 | −0.00 | 0.9804 | 0.9809 | +0.06 |
| XLM-R, LoRA LAPT (combined), LoRA FT | 0.9979 | 0.9982 | +0.03 | 0.9787 | 0.9791 | +0.04 |
| XLM-R, no LAPT, full FT | 0.9987 | 0.9986 | −0.01 | 0.9783 | 0.9791 | +0.09 |
| XLM-R, no LAPT, LoRA FT | 0.9974 | 0.9977 | +0.03 | 0.9783 | 0.9782 | −0.00 |
| TF-IDF (char) | 0.9974 | 0.9977 | +0.03 | 0.5000* | 0.5000* | 0.00* |
| TF-IDF (word) | 0.9974 | 0.9973 | −0.01 | 0.5000* | 0.5000* | 0.00* |

All 22 configurations evaluated on the same repetition-matched subset ($n$ = 1,102 pairs, 2,204 rows), using each model's original decision threshold without recalibration. The largest change in the RAID Diff column is bolded. *TF-IDF's original RAID threshold was unreachable (Section 6.1); the RAID-calibrated comparison is included for completeness but is not a meaningful indicator for these two rows (see Section 6.6).

**Table 7: Full test set vs. linguistic-marker-distribution-matched-subset accuracy for all 22 evaluated configurations, at both the default (0.5) and RAID-calibrated thresholds. Differences are reported in percentage points (pp). Unlike the length and repetition confound checks, this matched subset does not preserve human/machine pairs (see Section 5.2.4).**

| Configuration | Orig. Acc. | Matched Acc. | Diff (pp) | Orig. RAID Acc. | Matched RAID Acc. | RAID Diff (pp) |
|---|---|---|---|---|---|---|
| mBERT, full LAPT (combined), full FT | 0.9983 | 0.9981 | −0.02 | 0.9783 | 0.9731 | −0.52 |
| mBERT, full LAPT (combined), LoRA FT | 0.9970 | 0.9962 | −0.09 | 0.9825 | 0.9712 | −1.14 |
| mBERT, full LAPT (xho-only), full FT | 0.9983 | 0.9981 | −0.02 | 0.9680 | 0.9442 | −2.38 |
| mBERT, full LAPT (xho-only), LoRA FT | 0.9983 | 0.9962 | −0.21 | 0.9817 | 0.9635 | −1.82 |
| mBERT, full LAPT (zul-only), full FT | 0.9974 | 0.9962 | −0.13 | 0.9744 | 0.9596 | −1.48 |
| mBERT, full LAPT (zul-only), LoRA FT | 0.9970 | 0.9981 | +0.11 | 0.9783 | 0.9615 | −1.67 |
| mBERT, LoRA LAPT (combined), full FT | 0.9991 | 0.9981 | −0.11 | 0.9970 | 0.9962 | −0.09 |
| mBERT, LoRA LAPT (combined), LoRA FT | 0.9979 | 0.9962 | −0.17 | 0.9783 | 0.9615 | −1.67 |
| mBERT, no LAPT, full FT | 0.9974 | 0.9942 | −0.32 | 0.9757 | 0.9481 | **−2.76** |
| mBERT, no LAPT, LoRA FT | 0.9974 | 0.9923 | −0.51 | 0.9727 | 0.9558 | −1.70 |
| XLM-R, full LAPT (combined), full FT | 0.9996 | 1.0000 | +0.04 | 0.9774 | 0.9596 | −1.78 |
| XLM-R, full LAPT (combined), LoRA FT | 0.9979 | 0.9942 | −0.36 | 0.9680 | 0.9519 | −1.61 |
| XLM-R, full LAPT (xho-only), full FT | 0.9996 | 1.0000 | +0.04 | 0.9940 | 0.9904 | −0.36 |
| XLM-R, full LAPT (xho-only), LoRA FT | 0.9970 | 0.9923 | −0.47 | 0.9744 | 0.9519 | −2.25 |
| XLM-R, full LAPT (zul-only), full FT | 0.9987 | 0.9962 | −0.26 | 0.9736 | 0.9596 | −1.40 |
| XLM-R, full LAPT (zul-only), LoRA FT | 0.9983 | 0.9962 | −0.21 | 0.9714 | 0.9769 | **+0.55** |
| XLM-R, LoRA LAPT (combined), full FT | 0.9996 | 1.0000 | +0.04 | 0.9804 | 0.9654 | −1.50 |
| XLM-R, LoRA LAPT (combined), LoRA FT | 0.9979 | 0.9942 | −0.36 | 0.9787 | 0.9673 | −1.14 |
| XLM-R, no LAPT, full FT | 0.9987 | 1.0000 | +0.13 | 0.9783 | 0.9615 | −1.67 |
| XLM-R, no LAPT, LoRA FT | 0.9974 | 0.9923 | −0.51 | 0.9783 | 0.9731 | −0.52 |
| TF-IDF (char) | 0.9974 | 0.9923 | −0.51 | 0.5000* | 0.5000* | 0.00* |
| TF-IDF (word) | 0.9974 | 0.9904 | **−0.70** | 0.5000* | 0.5000* | 0.00* |

All 22 configurations evaluated on the linguistic-marker-matched subset ($n$ = 520, 260 human / 260 machine, unpaired), using each model's original decision threshold without recalibration. The largest change in each diff column is bolded. *TF-IDF's original RAID threshold was unreachable (Section 6.1); the RAID-calibrated comparison is included for completeness but is not a meaningful indicator for these two rows.

**Table 8: All comparisons (out of 100 tested, at the RAID-calibrated threshold) that remain significant after Bonferroni correction ($\alpha = 0.0005$). No comparisons were significant at the default threshold. Accuracy difference is model B minus model A.**

| Group | Model A | Model B | Accuracy diff. | $p$-value | Better |
|---|---|---|---|---|---|
| LAPT vs No-LAPT (mBERT) | mBERT, no LAPT, full FT | mBERT, LoRA LAPT, full FT | +0.0213 | $5.8 \times 10^{-11}$ | mBERT, LoRA LAPT |
| LAPT vs No-LAPT (XLM-R) | XLM-R, no LAPT, full FT | XLM-R, full LAPT (xho), full FT | +0.0158 | $1.9 \times 10^{-8}$ | XLM-R, full LAPT (xho) |
| Full LAPT vs LoRA LAPT | mBERT, full LAPT, full FT | mBERT, LoRA LAPT, full FT | +0.0188 | $1.2 \times 10^{-9}$ | mBERT, LoRA LAPT |
| Cross-lingual (mBERT) | mBERT, full LAPT (combined) | mBERT, full LAPT (xho) | −0.0102 | 0.0053 | mBERT, full LAPT (combined) |
| Cross-lingual (XLM-R) | XLM-R, full LAPT (combined) | XLM-R, full LAPT (xho) | +0.0166 | $6.8 \times 10^{-9}$ | XLM-R, full LAPT (xho) |
| Cross-lingual (XLM-R) | XLM-R, full LAPT (xho) | XLM-R, full LAPT (zul) | −0.0205 | $3.4 \times 10^{-10}$ | XLM-R, full LAPT (xho) |
| Backbone (with LAPT) | mBERT, full LAPT, LoRA FT | XLM-R, full LAPT, LoRA FT | −0.0145 | $1.5 \times 10^{-4}$ | mBERT |
| Backbone (with LAPT) | mBERT, LoRA LAPT, full FT | XLM-R, LoRA LAPT, full FT | −0.0166 | $6.8 \times 10^{-9}$ | mBERT |
| Backbone (with LAPT) | mBERT, full LAPT (xho), full FT | XLM-R, full LAPT (xho), full FT | +0.0260 | $2.2 \times 10^{-12}$ | XLM-R |
| Classifier LoRA vs Full | mBERT, LoRA LAPT, full FT | mBERT, LoRA LAPT, LoRA FT | −0.0188 | $5.4 \times 10^{-10}$ | mBERT, full FT |
| Classifier LoRA vs Full | mBERT, full LAPT (xho), full FT | mBERT, full LAPT (xho), LoRA FT | +0.0136 | $3.1 \times 10^{-4}$ | mBERT, LoRA FT |
| Classifier LoRA vs Full | XLM-R, full LAPT (xho), full FT | XLM-R, full LAPT (xho), LoRA FT | −0.0196 | $1.8 \times 10^{-9}$ | XLM-R, full FT |